

Pointer Analysis (2)

CSE552 Program Analysis — Lecture 15

Jaemin Hong

Flow-Sensitive Pointer Analysis

- ▶ Flow-insensitive pointer analysis is sometimes too imprecise

```
x = &a;  
*x = 1;  
x = &b;  
*x = 2;
```

- ▶ Flow-sensitive pointer analysis provides better precision

Combining Pointer Analysis with Another Analysis

- ▶ Pointer analysis can be performed as a pre-analysis, and the points-to information can be utilized in the main analysis
- ▶ However, performing pointer analysis as a part of the main analysis can provide additional precision

Abstract Domains

- ▶ $Cell = Var \cup Alloc$
- ▶ $Value = Interval \times \mathcal{P}(Cell)$
 - ▶ $([l, h], \{c, \dots\})$ represents an integer between l and h or a pointer to c
- ▶ $State = Cell \rightarrow Value$
- ▶ $JOIN(v) = \bigsqcup_{u \in pred(v)} \llbracket u \rrbracket$

Transfer Functions — Pointer-Irrelevant

- ▶ $x = e: t_v(\sigma) = \sigma[x \mapsto \text{eval}(\sigma, e)]$
- ▶ $\text{eval}(\sigma, x) = \sigma(x)$
- ▶ $\text{eval}(\sigma, n) = ([n, n], \perp)$
- ▶ $\text{eval}(\sigma, \text{input}()) = (\top, \perp)$
- ▶ $\text{eval}(\sigma, e_1 \text{ op } e_2) = \widehat{op}(\text{eval}(\sigma, e_1), \text{eval}(\sigma, e_2))$

Transfer Functions — Pointer-Relevant

- ▶ $\text{eval}(\sigma, \&x) = (\perp, \{x\})$
- ▶ $\text{eval}(\sigma, \text{alloc}()) = (\perp, \{\text{alloc-}i\})$
- ▶ $\text{eval}(\sigma, *e) = \text{fetch}(\sigma, \text{eval}(\sigma, e))$

- ▶ $\text{fetch}(\sigma, (-, C)) = \bigsqcup_{c \in C} \sigma(c)$

Transfer Functions — Example

```
x = 1;
// x: ([1, 1], bot)
y = 2;
// x: ([1, 1], bot), y: ([2, 2], bot)
if ... {
    z = &x;
    // x: ([1, 1], bot), y: ([2, 2], bot), z: (bot, {x})
} else {
    z = &y;
    // x: ([1, 1], bot), y: ([2, 2], bot), z: (bot, {y})
}
// x: ([1, 1], bot), y: ([2, 2], bot), z: (bot, {x, y})
w = *z;
// x: ([1, 1], bot), y: ([2, 2], bot), z: (bot, {x, y}), w:
    ([1, 2], bot)
```

Indirect Assignment (Incorrect)

- ▶ $*e_1 = e_2$: $t_v(\sigma) = \text{update}(\sigma, \text{eval}(\sigma, e_1), \text{eval}(\sigma, e_2))$
- ▶ $\text{update}(\sigma, (-, C), v) = \sigma[c_1 \mapsto v, \dots, c_n \mapsto v]$
where $C = \{c_1, \dots, c_n\}$

Indirect Assignment (Incorrect) — Example 1

```
x = 1;  
// x: ([1, 1], bot)  
y = &x;  
// x: ([1, 1], bot), y: (bot, {x})  
*y = 2;  
// x: ([2, 2], bot), y: (bot, {x})
```

Indirect Assignment (Incorrect) — Example 2

```
x = 1;
// x: ([1, 1], bot)
y = 2;
// x: ([1, 1], bot), y: ([2, 2], bot)
if ... {
  z = &x;
  // x: ([1, 1], bot), y: ([2, 2], bot), z: (bot, {x})
} else {
  z = &y;
  // x: ([1, 1], bot), y: ([2, 2], bot), z: (bot, {y})
}
// x: ([1, 1], bot), y: ([2, 2], bot), z: (bot, {x, y})
*z = 3;
// x: ([3, 3], bot), y: ([3, 3], bot), z: (bot, {x, y})
```

- ▶ Unsound results: [3,3] for x and y
 - ▶ However, x may be 1, and y may be 2

Indirect Assignment (Still Incorrect)

- ▶ $*e_1 = e_2$: $t_v(\sigma) = \text{update}(\sigma, \text{eval}(\sigma, e_1), \text{eval}(\sigma, e_2))$
- ▶ $\text{update}(\sigma, (-, C), v) = \begin{cases} \sigma[c \mapsto v] & \text{if } C = \{c\} \\ \bigsqcup_{c \in C} \sigma[c \mapsto \sigma(c) \sqcup v] & \text{otherwise} \end{cases}$
- ▶ Strong update: the new value overwrites the memory
- ▶ Weak update: the new value is joined with the old value

Indirect Assignment (Still Incorrect) — Example 1

```
x = 1;
// x: ([1, 1], bot)
y = 2;
// x: ([1, 1], bot), y: ([2, 2], bot)
if ... {
  z = &x;
  // x: ([1, 1], bot), y: ([2, 2], bot), z: (bot, {x})
} else {
  z = &y;
  // x: ([1, 1], bot), y: ([2, 2], bot), z: (bot, {y})
}
// x: ([1, 1], bot), y: ([2, 2], bot), z: (bot, {x, y})
*z = 3;
// x: ([1, 3], bot), y: ([2, 3], bot), z: (bot, {x, y})
```

Indirect Assignment (Still Incorrect) — Example 2

```
for i in [0, 1] {  
  x = alloc();  
  // x: (bot, {alloc-0})  
  *x = 0;  
  // x: (bot, {alloc-0}), alloc-0: ([0, 0], bot)  
  if i == 0 {  
    y = x;  
    // x: (bot, {alloc-0}), y: (bot, {alloc-0}), alloc-0:  
      ([0, 0], bot)  
  }  
  // x: (bot, {alloc-0}), y: (bot, {alloc-0}), alloc-0: ([0,  
    0], bot)  
}  
// x: (bot, {alloc-0}), y: (bot, {alloc-0}), alloc-0: ([0,  
  0], bot)  
*x = 1;  
// x: (bot, {alloc-0}), y: (bot, {alloc-0}), alloc-0: ([1,  
  1], bot)
```

- ▶ Unsound results: $[1, 1]$ for alloc-0
 - ▶ However, $*y$ is 0

Indirect Assignment (Correct)

- ▶ $x = e$: $t_v(\sigma) = \text{update}(\sigma, (\perp, \{x\}), \text{eval}(\sigma, e))$
- ▶ $*e_1 = e_2$: $t_v(\sigma) = \text{update}(\sigma, \text{eval}(\sigma, e_1), \text{eval}(\sigma, e_2))$
- ▶
$$\text{update}(\sigma, (-, C), v) = \begin{cases} \sigma[x \mapsto v] & \text{if } C = \{x\} \\ \bigsqcup_{c \in C} \sigma[c \mapsto \sigma(c) \sqcup v] & \text{otherwise} \end{cases}$$
- ▶ Strong update: the new value overwrites the memory
 - ▶ Used when the abstract cell represents a single runtime cell
- ▶ Weak update: the new value is joined with the old value
 - ▶ Used when the abstract cell represents possibly multiple runtime cells

Indirect Assignment (Correct) — Example

```
for i in [0, 1] {  
  x = alloc();  
  // x: (bot, {alloc-0})  
  *x = 0;  
  // x: (bot, {alloc-0}), alloc-0: ([0, 0], bot)  
  if i == 0 {  
    y = x;  
    // x: (bot, {alloc-0}), y: (bot, {alloc-0}), alloc-0:  
      ([0, 0], bot)  
  }  
  // x: (bot, {alloc-0}), y: (bot, {alloc-0}), alloc-0: ([0,  
    0], bot)  
}  
// x: (bot, {alloc-0}), y: (bot, {alloc-0}), alloc-0: ([0,  
  0], bot)  
*x = 1;  
// x: (bot, {alloc-0}), y: (bot, {alloc-0}), alloc-0: ([0,  
  1], bot)
```

Interprocedural Analysis — Without Pointers

$x = f(e_1, \dots, e_n)$:

$$t_v(\sigma) = \left\{ \begin{array}{l} ([x_1 \mapsto \text{eval}(e_1, \sigma), \dots, x_n \mapsto \text{eval}(e_n, \sigma)], \text{entry}(f)), \\ (\sigma[x \mapsto \sigma_{\text{return}}(\text{RET})], \text{after}(v)) \end{array} \right\}$$

where

- ▶ $x_1, \dots, x_n$ are the parameters of f
- ▶ $\text{entry}(f)$ is the entry node of f
- ▶ σ_{return} is the state at the return node of f
- ▶ $\text{after}(v)$ is the after-call node

return:

$$t_v(\sigma) = \{(\sigma_{v_1}[x_1 \mapsto \sigma(\text{RET})], \text{after}(v_1)), \dots\}$$

where

- ▶ v_i is a call node
- ▶ σ_{v_i} is the state at v_i
- ▶ x_i is the variable assigned the return value of v_i
- ▶ $\text{after}(v_i)$ is the after-call node for v_i

Interprocedural Analysis — With Pointers

- ▶ The values of address-taken variables and heap addresses should be passed across function boundaries as well

$x = f(e_1, \dots, e_n)$:

$$t_v(\sigma) = \left\{ \begin{array}{l} ([x_1 \mapsto \text{eval}(e_1, \sigma), \dots, c_1 \mapsto \sigma(c_1), \dots], \text{entry}(f)), \\ (\sigma[x \mapsto \sigma_{\text{return}}(\text{RET}), c_1 \mapsto \sigma_{\text{return}}(c_1), \dots], \text{after}(v)) \end{array} \right\}$$

where

- ▶ c_i is an address-taken variable or a heap address

return:

$$t_v(\sigma) = \{(\sigma_{v_1}[x_1 \mapsto \sigma(\text{RET}), c_1 \mapsto \sigma(c_1), \dots], \text{after}(v_1)), \dots\}$$

Interprocedural Analysis — Example

```
fn foo(p) {  
    // x: ([0, 0], bot), p: (bot, {x})  
    *p = 1;  
    // x: ([1, 1], bot), p: (bot, {x})  
}  
  
x = 0;  
// x: ([0, 0], bot)  
y = &x;  
// x: ([0, 0], bot), y: (bot, {x})  
foo(y);  
// x: ([1, 1], bot), y: (bot, {x})
```

Recursive Functions

- ▶ When x is a variable of a (mutually) recursive function, multiple runtime cells denoted by the abstract cell x can coexist

Recursive Functions — Example

```
fn foo() {  
  let x = 0;  
  // x: ([0, 0], bot)  
  let p = &x; ...  
  if ... {  
    // x: ([0, 0], bot)  
    foo();  
    // x: ([1, 1], bot)  
  } else {  
    x = 1;  
    // x: ([1, 1], bot)  
  }  
  // x: ([1, 1], bot)  
}  
  
foo();
```

- ▶ Unsound results: $[1, 1]$ for x after `foo` returns
 - ▶ However, x is 0

Updates in Interprocedural Analysis

$$\text{update}(\sigma, (-, C), v) = \begin{cases} \sigma[x \mapsto v] & \text{if } C = \{x\} \wedge x \text{ is a local} \\ & \text{variable of a non-recursive} \\ & \text{function} \\ \bigsqcup_{c \in C} \sigma[c \mapsto \sigma(c) \sqcup v] & \text{otherwise} \end{cases}$$

- ▶ Strong update: the new value overwrites the memory
 - ▶ Used when the abstract cell represents a single runtime cell
- ▶ Weak update: the new value is joined with the old value
 - ▶ Used when the abstract cell represents possibly multiple runtime cells

Updates in Interprocedural Analysis — Example

```
fn foo() {  
  let x = 0;  
  // x: ([0, 0], bot)  
  let p = &x; ...  
  if ... {  
    // x: ([0, 0], bot)  
    foo();  
    // x: ([0, 1], bot)  
  } else {  
    x = 1;  
    // x: ([0, 1], bot)  
  }  
  // x: ([0, 1], bot)  
}  
  
foo();
```

Summary

- ▶ Flow-sensitive pointer analysis provides better precision than flow-insensitive analysis
- ▶ Indirect assignment through a pointer requires a case split: strong update overwrites the cell, while weak update joins with the old value
- ▶ Strong update is sound only when the abstract cell denotes a single runtime cell; otherwise weak update must be used
- ▶ Local variables of (mutually) recursive functions may denote multiple coexisting runtime cells, so strong update is restricted to local variables of non-recursive functions